

L04: Error Handling, Testing, I/O, and Data Files

Dr. Todd Jones

t.r.jones@reading.ac.uk

Department of Computer Science

Lecture Objectives

- Understand the concept of an **exception** in Python.
- Comprehend and apply different strategies to **handle exceptions**.
- Acknowledge the **different kinds of exceptions** that we can encounter.
- Understand the **purposes and benefits** of exceptions.
- Articulate why **testing** is essential in professional software development.
- Demonstrate the core principles of **Test-Driven Development**, and compare the usage of `unittest` and `pytest` in creating reliable test suites.
- Handle file operations through both basic tools and **context managers**.
- Explore reading and writing **JSON** data, appreciating how this format supports structured configuration and data exchange.
- Explore **command-line interfaces** using both `sys.argv` and `argparse`, constructing small, user-friendly tools for data processing.

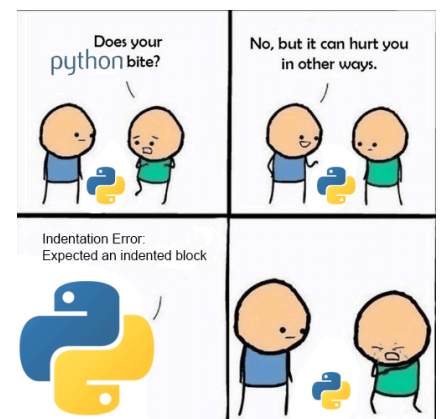
Integrating these techniques to design modular, testable, and maintainable Python programs, **reflects best professional practices**.

Exceptions and Errors

"**Writing code**" is about making code that *runs well*.

"**Software engineering**" is about making code that *fails well*.

- **Syntax errors** arise as the text of the code is processed and the interpreter finds that a certain statement is at odds with the prescribed rules of usage.
- **Runtime errors** occur while the code is executing.
(Think: dividing by zero.)
- In Python, these are known as **exceptions**.
 - Exceptions are triggered automatically on errors.
 - They can be intercepted by the code.



```
In [ ]: # Create a syntax error (Improper if-statement. No colon.)
```

```
if 3 < 1 print("That's really weird!")
```

```
In [ ]: # Create a syntax error (Improper use of indentation.)
```

```
if 3 < 1:
print("That's really weird!")
```

```
In [ ]:
```

```
'''
Create a runtime error (exception, because we cannot concatenate strings
and integers.)

We can tell that it is an exception, as opposed to a syntax error,
in the Jupyter Notebook because it creates a "Traceback".

The kind of exception we get is a "TypeError".
'''

my_new_string = "This failed string would have " + 10 + \
                " words if this worked."
```

Why do we use exceptions?

- In a nutshell, exceptions let us **jump out** of arbitrarily large chunks of code.

- Let's consider a pizza-making process.
- To make a pizza and deliver it, we need to go through several steps or stages:
 1. Think up the recipe.
 2. Obtain the ingredients.
 3. Prepare the dough.
 4. Add the toppings.
 5. Bake the pie.
 6. ...



- Imagine that there is a problem in the **Prepare the dough** sub-process (*a function*).
- In this case, we would like to have our attention pulled to the instruction (*the code*) that is relevant.
- What would we like to have happen during a failure of the process?
 - The oven caught fire and the pizza shop is filled with smoke.
 - The delivery driver got lost.
 - The mushrooms were all eaten by mice.
 - The mushrooms were all eaten by the chef, who has a life-threatening allergy to mushrooms.
- In our code, we might have to **care about which kinds of exceptions are happening and where they are happening**.
 - Maybe division by zero makes the result meaningless; maybe it can be ignored or handled.

```
In [ ]: # Basic Python division cares about dividing by zero and will
        # raise an exception.
        res = 5 / 0
```

```
In [ ]: # When performing division with numpy, we only get --warnings-- when
        # we divide by zero.

        # The code can otherwise proceed without stopping.

        # Numpy acknowledges that the developer might be OK with this result,
        # but should be made aware of it via a Warning.

import numpy as np

num = np.arange(10)
den = np.zeros(10)

res = num / den

print(res)
```

```
In [ ]: # If you are aware that these warnings are possible,
        # but don't care about them,
        # they can be ignored.

import numpy as np
```

```
# Suppress warnings:
#   divide: number / 0 --> inf
#   invalid: 0 / 0      --> nan
np.seterr(divide='ignore', invalid='ignore')

# Generate some values
num = np.arange(10)
den = np.zeros(10)

# Apply operation
res = num / den

# Check result
print(res)
```

Roles of Exceptions

Error Handling

- By default, on error, Python will **stop the program and display an error message**.
- As an alternative to the default behaviour, we can use the `try:` clause to define **contingencies** in the event of an exception:

```
try:
    <expression>
except:
    <expression>
else:
    <expression>
finally:
    <expression>
```

- If an exception occurs during the `try:` clause, your program can **resume execution** and proceed to perform some set of specified actions.
 - In this case, any code under `except:` (the *except clause*) *might* be executed.
- **Only** in the absence of an exception, any code under `else:` would be executed.

Event Notification

- Exceptions can be used to **signal valid conditions**, perhaps in the `except:` clause. Certain failures might be a valid result.
- For example, a search function might raise an exception when the value is not found instead of returning some coded result flag that would require further manual processing.

Special-case handling

- Certain exceptions might occur very rarely.
- Instead of designing your code to handle a special case in multiple places, an exception handler could be used in **higher levels** of a program.

- Similarly, an `assert` statement can be useful to check on special cases, particularly during code development.

Termination actions

- When including a `finally:` clause in association with a `try:` clause, we can ensure that any required end-of-program tidying (e.g. writing messages, closing files, etc.) is performed, regardless of whether an exception is encountered.

Unusual control flows

- Python does not have a "go to" statement to break sequential execution.
- Exceptions can be used as a sort of high-level, structured "go to", allowing you to implement more exotic control flows.
- For example, the `raise` statement can be used to jump out of multiple loops or deeply nested calls without a scattering of conditional checks throughout the code.

Possible Exceptions

The following table lists *important* built-in exceptions in Python.

- This list is not exhaustive.

Source: <https://www.tutorialsteacher.com/python/error-types-in-python>

Exception	Description
AssertionError	Raised when the assert statement fails.
AttributeError	Raised on the attribute assignment or reference fails.
EOFError	Raised when the input() function hits the end-of-file condition.
FloatingPointError	Raised when a floating point operation fails.
GeneratorExit	Raised when a generator's close() method is called.
ImportError	Raised when the imported module is not found.
IndexError	Raised when the index of a sequence is out of range.
KeyError	Raised when a key is not found in a dictionary.
KeyboardInterrupt	Raised when the user hits the interrupt key (Ctrl+c or delete).
MemoryError	Raised when an operation runs out of memory.
NameError	Raised when a variable is not found in the local or global scope.
NotImplementedError	Raised by abstract methods.
OSError	Raised when a system operation causes a system-related error.
OverflowError	Raised when the result of an arithmetic operation is too large to be represented.
ReferenceError	Raised when a weak reference proxy is used to access a garbage collected referent.
RuntimeError	Raised when an error does not fall under any other category.

Exception	Description
StopIteration	Raised by the next() function to indicate that there is no further item to be returned by the iterator.
SyntaxError	Raised by the parser when a syntax error is encountered.
IndentationError	Raised when there is an incorrect indentation.
TabError	Raised when the indentation consists of inconsistent tabs and spaces.
SystemError	Raised when the interpreter detects internal error.
SystemExit	Raised by the sys.exit() function.
TypeError	Raised when a function or operation is applied to an object of an incorrect type.
UnboundLocalError	Raised when a reference is made to a local variable in a function or method, but no value has been bound to that variable.
UnicodeError	Raised when a Unicode-related encoding or decoding error occurs.
UnicodeEncodeError	Raised when a Unicode-related error occurs during encoding.
UnicodeDecodeError	Raised when a Unicode-related error occurs during decoding.
UnicodeTranslateError	Raised when a Unicode-related error occurs during translation.
ValueError	Raised when a function gets an argument of correct type but improper value.
ZeroDivisionError	Raised when the second operand of a division or module operation is zero.

Hierarchy of Exceptions in Python

In Python, exceptions are organized into a **hierarchical structure** of classes, allowing for specific and generalised exception handling.

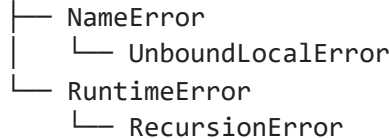
At the top of this hierarchy is the `BaseException` class.

Extract of common exceptions (<https://docs.python.org/3/library/exceptions.html#exception-hierarchy>):

```

BaseException
├── SystemExit
├── KeyboardInterrupt
├── GeneratorExit
├── Exception
│   ├── ArithmeticError
│   │   ├── ZeroDivisionError
│   │   ├── OverflowError
│   │   └── FloatingPointError
│   ├── LookupError
│   │   ├── IndexError
│   │   └── KeyError
│   ├── ImportError
│   │   └── ModuleNotFoundError
│   ├── OSError
│   │   ├── FileNotFoundError
│   │   └── PermissionError
│   ├── ValueError
│   │   └── UnicodeError
│   │       ├── UnicodeEncodeError
│   │       └── UnicodeDecodeError
│   └── TypeError

```



Key Points:

- `BaseException` : The **root** of the exception hierarchy. All exceptions inherit from this class.
- `Exception` : The base class for all built-in, non-system-exiting exceptions.
- `SystemExit` , `KeyboardInterrupt` , and `GeneratorExit` : Inherit directly from `BaseException` and are used for control-flow exceptions.
- This hierarchy allows you to catch exceptions at different levels of **specificity**.

```
In [ ]: # We can print built-in exceptions, functions, and attributes.

# The dir() function returns all properties and methods of the
#   specified object, without the values.

# The locals() function returns an updated dictionary of the current
#   local symbol table.

# We examine the properties and methods of the <module 'builtins' (built-in)>

print(dir(locals()['__builtins__']))
```

```
In [ ]: # Examining the 'ArithmeticError' exception, as an example,
#   we see that it is its own "type" with a variety of properties.

print(type(locals()['__builtins__'].ArithmeticError))
dir(locals()['__builtins__'].ArithmeticError)
```

Ways to raise, catch, and handle exceptions

We will demonstrate how to **raise, catch, and handle** exceptions through a series of examples.

```
In [ ]: # Let's define a function that returns the contents of a
#   specified element of a supplied object (e.g., a string, list, array, etc.)

def fetcher(obj, index):
    return obj[index]
```

```
In [ ]: # Create an object, in this case a string.

x = 'Lupita'
```

```
In [ ]: # Let's explore the different kinds of exceptions we can get while
#   executing the function.

# No error:
fetcher(x, 5)
```

```
In [ ]: # IndexError:
fetcher(x, 9)
```

```
In [ ]: # ValueError:
fetcher(x, int('m'))
```

```
In [ ]: # Correction of the ValueError
        fetcher(x, int('3'))
```

Remember, by default, a raised exception will cause the code to stop in a failed state.

Let's try to catch and handle the exceptions that are being raised by creating a function that calls the `fetcher()` .

```
In [ ]: # Defining a "catcher" to handle the exceptions
        # -- BUT this one does NOT handle exceptions yet!

        # The input() function allows user input.

        def catcher():
            word = str(input("Please enter the word to evaluate: "))
            indx = int(input("Please enter the position of the letter to retrieve: "))
            print('The letter is: ',
                  fetcher(word,
                           indx)) # This is the line that will trigger the exception.
```

```
In [ ]: # Simulating an index outside the limit of the string length (IndexError)
        catcher()
```

```
In [ ]: # Simulating a wrong kind of index (e.g., give it a character instead of an integer, ValueError)
        catcher()
```

Handling Exceptions

Simple exceptions, `try` and `except`:

- Under `try:` , all statements are executed until the clause ends or an exception is reached.
- If an exception is encountered during a `try:` , one or more `except:` statements can be used to catch and handle the exception(s).
 - Notice that exception handlers can handle exceptions even when they occur inside functions that are called within the `try:` .

```
In [ ]: # Simple exception handling
        def catcher_exp():

            word = str(input("Please enter the word to evaluate: "))

            try:
                indx = int(
                    input("Please enter the position of the letter to retrieve: "))

                print('The letter is: ', fetcher(
                    word, indx)) # This is the line that will trigger the exception
            except:
                print('There was an exception!')
```

```
In [ ]: catcher_exp()
```

- When an exception occurs in a program running the above function, the program will **continue as well as inform you** about the fact that the function call was not successful.
 - What **you did not get to see above** was the type of error that was thrown as a result of the function call.
 - This can easily **mask** the true nature of the programming error.
 - Use caution.
 - In order to see exactly what went wrong, you would **need to catch the error** that the code threw.
-

```
In [ ]: # Exception handling with additional detail

def catcher_exp_det():
    word = str(input("Please enter the word to evaluate: "))
    try:
        indx = int(input("Please enter the position of the letter to retrieve: "))
        print('The letter is: ',
              fetcher(word, indx)) # This is the line that will trigger the exception
    except Exception as ex:
        print('There was an exception: ', ex)
```

```
In [ ]: # Incorrect index value

catcher_exp_det()
```

```
In [ ]: # Incorrect index type

catcher_exp_det()
```

Note: By adding a simple `raise` statement within an `except` block, we can "re-raise" the caught exception. We might do this to add additional functionality or messaging before ensuring a program fails with the original error.

Handling *specific* exceptions (`try` and `except`):

```
In [ ]: # In this case, we are going to handle the SPECIFIC case when the index
#        is out of range (IndexError)

# Because we only specify one exception type, any others will not be
# explicitly handled.

def catcher_exp_indx():
    word = str(input("Please enter the word to evaluate: "))
    try:
        indx = int(input("Please enter the position of the letter to retrieve: "))
        print('The letter is: ',
              fetcher(word, indx)) # This is the line that will trigger the exception
    except IndexError:
        print('The index you provided is out of the limit.')
```

```
In [ ]: catcher_exp_indx()
```

```
In [ ]: # Handle TWO SPECIFIC exceptions out of range and wrong index:

def catcher_exp_indx_val():
    word = str(input("Please enter the word to evaluate: "))
    try:
        indx = int(input("Please enter the position of the letter to retrieve: "))
        print('The letter is: ', fetcher(word, indx))
    except IndexError:
        print('The index you provided is out of the limit.')
    except ValueError:
        print('The index you provided is of the wrong type.')
```

```
In [ ]: catcher_exp_indx_val()
```

NOTE: An except clause may name multiple exceptions as a tuple:

```
...
except (RuntimeError, TypeError, NameError, ValueError):
    print('Message...')
```

```
In [ ]: # NOTE: The `except` clauses don't know anything about variables modified
#         or created in the `try`.

# Let's handle out of range and wrong index, while providing more information
# (remind user what they supplied):

# Note: in comparison to the previous catcher, we've moved the indx input
# before the `try:` and left the conversion to int within the try.

def catcher_exp_indx_val_more():
    word = str(input("Please enter the word to evaluate: "))
    indx = input("Please enter the position of the letter to retrieve: ")
    try:
        indx = int(indx)
        print('The letter is: ', fetcher(word, indx))
    except IndexError:
        print(f'The index you provided, "{indx}", is out of the limit.')
    except ValueError:
        print(f'The index you provided, "{indx}", is of the wrong type.')
```

```
In [ ]: catcher_exp_indx_val_more()
```

Handling particular exceptions (try, except, else, and finally):

- The **else:** construct, paired with **try:**, contains code to be executed **ONLY** when the **try:** succeeds and there are **NO** exceptions under **try:**.
- This separation helps us to avoid unintentionally catching exceptions from code that is not supposed to be inspected for exceptions as defined by the **except** s.
- If we placed the same code after the full **try** construct, it could be executed if an exception was found, and this could go against our intentions.

```
In [ ]: # Add an `else` condition to execute ONLY when there is no exception.

# Note: The catcher has been modified to run within an INFINITE LOOP,
#         so the user can have multiple tries.
```

```
def catcher_exp_indx_val_els():

    word = str(input("Please enter the word to evaluate: "))

    while True:          # Infinite Loop.  New addition to the catcher routine.
        try:
            indx = int(input("Please enter the position of the letter to retrieve: "))
            ltr = fetcher(word, indx) # This is the line that will trigger the exception
        except IndexError:
            print('The index you provided is out of the limit.')
        except ValueError:
            print('The index you provided is of the wrong type.')
        else:
            print('The letter is: ', ltr)
            break
```

In []: catcher_exp_indx_val_els()

Add in `finally` behaviour

- This section of code will **ALWAYS** be run, with or without any previously encountered exceptions.
- This can be useful for tidying up (e.g. closing open files properly).
 - Alternatively, for specific kinds of object processing, the use of a **context manager**, such as the `with / as` construct, can guarantee that termination actions occur, without regard for any exceptions within its clause.

In []: *# Multiple tries are allowed, as before.*

```
def catcher_exp_indx_val_els_fin():
    word = str(input("Please enter the word to evaluate: "))
    while True:
        try:
            indx = int(input("Please enter the position of the letter to retrieve: "))
            ltr = fetcher(word, indx) # This is the line that will trigger the exception
        except IndexError:
            print('The index you provided is out of the limit.')
        except ValueError:
            print('The index you provided is of the wrong type.')
        else:
            print('The letter is: ', ltr)    # NOTE: The `else` DOES know about what happened
            break
        finally:
            print('I am always here.')      # NOTE: The `finally` MIGHT know about what happened
```

In []: catcher_exp_indx_val_els_fin()

Coarse vs Fine-Grained Exception Handling

Coarse-level Exception Handling	Fine-grained Exception Handling
Easy to implement; catches all errors indiscriminately	Requires thoughtful, explicit implementation

Coarse-level Exception Handling	Fine-grained Exception Handling
Risks masking important errors, complicating debugging	Enables clearer, targeted debugging and easier error tracing
Suitable mainly as a last-resort safety net	Preferred practice for robust, maintainable code
Should be used sparingly and cautiously	Recommended as the default approach

Via hybrid approaches (for robustness), you might introduce **layered** approaches (nested `try` s), starting with fine-grained exceptions at the deepest levels and adding a broader safety net higher up the call stack.

raise and assert:

- **The `raise` statement can be used to directly raise (i.e. "throw" or "force") an exception.**
- You can supply your own error message as an argument.

```
In [ ]: # Example:
# The catcher is modified to ensure that an exception is raised when the word does not consist
# The .isalpha() string method returns True if all the characters are alphabet letters (a-z).

def catcher_exp_idx_val_raise():
    word = str(input("Please enter the word to evaluate (just alphabet characters): "))
    if word.isalpha():
        while True:
            try:
                indx = int(input("Please enter the position of the letter to retrieve: "))
                print('The letter is: ', fetcher(word, indx))
                break
            except IndexError:
                print('The index you provided is out of the limit.')
            except ValueError:
                print('The index you provided is of the wrong type.')
        else:
            raise TypeError('Only alphabet characters are allowed.')
```

```
In [ ]: catcher_exp_idx_val_raise()
```

We can **create** our own custom exceptions, too, by **inheriting** from the `Exception` class.

```
In [ ]: # Pass the Exception class into the custom definition.
#         ↓
class MySpecialException(Exception):
    pass # Good enough to make a copy of Exception!
# def __init__(self, arg0): # To do more, note that Exceptions are
#     super().__init__(arg0) # initialised with a single argument.
#     self.arg0 = arg0 # We might extend the base Exception to
#                       # include attributes, methods, etc.

# Load sys package for system-specific parameters and functions
import sys

# python version check
if sys.version_info.major != 4:
    print(' STOP -- This code now works with Python 4 only.')
```

```

print('    On Machine-X, try: module load scitools')
print('    On Machine-Y, try: module load cray-python')
#     sys.exit()    # Since we've manually printed some guidance, a Python
                    # script might choose to simply stop without further
                    # messages.
raise MySpecialException('A bad thing happened.')

```

- **With an `assert` statement, we can assert that a certain condition is met before continuing with execution. Otherwise, an `AssertionError` is raised.**

```
assert (<condition>), "Error Message"
```

- Examples:
 - Your function performs division, but the denominator must be odd.
 - Your code should only be run on specific platforms.
 - Your code should only be run with specific versions of Python.

```
In [ ]: # Example: We `assert` that catcher_exp_idx_val_raise() only be run on Linux.
```

```

import sys
assert ('linux' in sys.platform), "This code runs on Linux only."
#assert ('win32' in sys.platform), "This code runs on Windows only."
catcher_exp_idx_val_raise()

```

```
In [ ]: # Example: python version check for Python 2 code.
# NOTE: Executing this cell will give a syntax error when using a Python 3 kernel.
```

```

assert (sys.version_info.major < 3), "This code only works with Python versions prior to 3."

# If we're using a lower version, like Python 2, then this code would be executed.
# print 'This print command works with Python 2.'
#
#                               # Printing became a function in Python 3.
# num = 3 / 2                  # Result is 1,
#                               # Python 2 defaults to integer division.
# print range(5)               # Result is [0, 1, 2, 3, 4]
#                               # Python 2's range() returns a list instead of an iterable

```

Exceptions Summary

- **Exceptions** may be raised by Python or by your own program.
- Uncaught exceptions eventually filter up to the default exception handler Python provides at the top of your program. This handler prints the familiar error message and shuts down your program.
- Exception processing is useful for **error handling, termination actions, and event notification**.
 - It can also simplify the handling of **special cases** and can be used to implement **alternative control flows** as a sort of structured "go to" operation.
 - In general, exception processing also cuts down on the amount of error-checking code your program may require. Because all errors filter up to handlers, you might not need to test the outcome of every operation.
- If you do not want the default message and shutdown, you can code `try / except` statements to **catch and recover from exceptions** that are raised within its nested code block.

- Once an exception is caught, the exception is terminated, and your program continues after the `try`.
 - The `try / finally` statement can be used to **ensure actions are run** after a block of code exits, regardless of whether the block raises an exception or not. The `with / as` statement can also be used to ensure termination actions are run, but only when processing object types that support it.
 - The `raise` and `assert` statements can be used to **trigger built-in or custom exceptions**, as if it had been raised by Python itself.
 - In principle, you can also raise an exception by making a programming mistake, but that is not usually an explicit goal!
-

Note that **much** more can be done here.

There is certainly the possibility to encounter a `TypeError`, a `ValueError`, or an `ArithmeticError` in various implementations of this kind of code.

Testing

Broken code can cause major, repetitive headaches in production.

- Bugs in production code can lead to lost data, unhappy users, or catastrophic failures in essential systems.

As well-tested code is easier to **maintain and extend**, because unintended side-effects are more likely to be caught, real-world software engineering relies on testing to **ensure reliability**.

- Most real-world development teams have some form of testing culture, often using **CI/CD** (Continuous Integration/Continuous Deployment) pipelines.

Through **Test-Driven Development (TDD)**, we can write tests first, then implement code to pass those tests.

Along the way we will rely on `assert` ions to ensure that expected conditions hold.

Types of Tests

Unit tests

- test **individual** units (functions/classes/modules)

Integration tests

- test how parts **interact** (a function that reads from a file and processes data)

System end-to-end tests

- validate the **overall behaviour** of an application, often involving command-line interfaces or external APIs

...and many other kinds of important bespoke testing, including user and scaling tests.

Test-Driven Development (TDD)

If you begin with a test that is known to **fail**, then when the test finally **passes**, you can be confident that the changes made are what **fixed** them.

Write **failing tests first**, then just enough code to **pass** them, and finally move on to **refactoring** for greater efficiency.

Framework:

1. RED: Write a new test; it fails initially.
2. GREEN: Implement code to make the test pass.
3. REFACTOR: Clean up the code without breaking the successful test.

This approach ensures a **clear definition** of what the code **should** do before the implementation stage, and enables **short, focused** development cycles that **catch bugs early**.

Common Python Testing Tools

`unittest` : <https://docs.python.org/3/library/unittest.html>

- Traditional, found in the Python standard library
- Uses a **class**-based approach

`pytest` : <https://docs.pytest.org/en/stable/>

- Supports execution of `unittest` cases
- Simpler **function**-based approach
- Plugin ecosystem available for popular tools, like the Django framework

Generally, programatic tests follow the **Arrange-Act-Assert** model:

- **Arrange**, or set up, the conditions for the test.

- **Act** by calling some function or method.
- **Assert** that some end condition is true.

Alternatively, this is referred to as the *Input, Execute, Assert* pattern.

```
In [ ]: def subtractor(m, n):
        pass
        # return m - n

def test_subtractor():
    a = 54
    b = 31
    assert subtractor(a, b) == 23, "Incorrect implementation of subtractor."

test_subtractor()
```

unittest

The `unittest` module is part of Python's **standard library**. It uses a **class-based** approach, where tests are defined as methods in subclasses of the `unittest.TestCase` class. *Mimics JUnit framework from Java.*

A **test suite** consists of a collection of multiple test cases to be run together.

A **test runner** executes tests and checks the assertions before reporting test results.

- This typically involves the `unittest.main()` entry point call.

Instead of the basic `assert` statement, this class structure relies on internally defined methods (most with oppositely named `...Not...` methods):

Method	Equivalent to
<code>.assertEqual(a, b)</code>	<code>a == b</code>
<code>.assertTrue(x)</code>	<code>bool(x) is True</code>
<code>.assertFalse(x)</code>	<code>bool(x) is False</code>
<code>.assertIs(a, b)</code>	<code>a is b</code>
<code>.assertIsNone(x)</code>	<code>x is None</code>
<code>.assertIn(a, b)</code>	<code>a in b</code>
<code>.assertIsInstance(a, b)</code>	<code>isinstance(a,b)</code>

Scenario:

We want to test a small math utility function.

```
In [ ]: '''
Often, unittest is run from the command line.

For demonstration in a Jupyter Notebook,
we have added the arguments to unittest.main().
Jupyter Notebook can pass extra command-line
```

arguments that can interfere with unittest's own argument parser. The argv construct below will override anything that Jupyter sends. exit=False ensures that unittest.main() does not call sys.exit() upon completion, which would shut down the notebook kernel.

If run as a script, other arguments may be passed and we would expect sys.exit() at the end of the run.

```
...

import unittest

# The code we want to test:
def add_numbers(a, b):
    """Return the sum of a and b."""
    return a + b

class TestMathUtils(unittest.TestCase):
    """Tests for the add_numbers function."""

    def test_add_positive_values(self):
        self.assertEqual(add_numbers(2, 3), 5)
        self.assertEqual(add_numbers(10, 5), 15)

    def test_add_negative_values(self):
        self.assertEqual(add_numbers(-1, -2), -3)
        self.assertEqual(add_numbers(-10, -5), -15)

    def test_add_zero(self):
        self.assertEqual(add_numbers(0, 5), 5)
        self.assertEqual(add_numbers(5, 0), 5)

# The following block is necessary to run tests in a notebook:
if __name__ == '__main__':
    unittest.main(argv=['first-arg-is-ignored'], exit=False)
```

Alternatively, this can be run as a script with the included file:

```
python .\demo_unittest.py
```

Or, we can run tests using the `unittest` module directly from the command line:

```
python -m unittest demo_unittest.py
```

- This implementation does not require that `unittest.main()` be written into the code.
- The `-m` flag indicates that we would like to run `unittest` as a script. The name of `demo_unittest.py` is an input argument to the `unittest` script.

```
In [ ]: !python -m unittest demo_unittest.py
```

Exercise

1. Write a new function: `multiply_numbers`.
2. Write a new test class, and add methods that verify the correct output for:
 - Positive numbers (e.g., `2 * 3 = 6`)
 - Negative numbers (e.g., `-1 * 3 = -3`)
 - Zero cases (e.g., `0 * 5 = 0`)
3. Run your tests in the notebook and demonstrate a failing scenario.

```
In [ ]: #???
```

pytest

`pytest` is a popular **third-party** library, providing a testing framework with a more **concise** approach (particularly for **fixture** creation) and expansion plugins to interface with specific frameworks.

- A **fixture** is a reusable setup that prepares and provides a consistent context or set of data that your tests can depend on.
 - Fixtures help to establish the necessary conditions for tests to run, such as initialising objects, configuring environments, or connecting to databases.

Noteworthy elements:

- Function-based tests
- Automatic discovery of tests
 - functions with `test_*` prefix
 - in modules with `test_*.py` prefix or `*_test.py` suffix
 - identifies and runs `unittest` tests
- Assertions use the plain `assert` statement, rather than specialised methods.

The `test_string_utils.py` module can be tested at the command line.

```
# Module to demonstrate use of pytest.
```

```
def reverse_string(s):  
    """Return the reverse of string s."""  
    return s[::-1]  
  
def test_reverse_string_simple():  
    assert reverse_string("abc") == "cba"  
  
def test_reverse_string_palindrome():  
    assert reverse_string("racecar") == "racecar"
```

```
In [ ]: !pytest test_string_utils.py
```

Comments on above output:

- `pluggy-1.5.0`: `pluggy` is the plugin-management framework that `pytest` itself uses to implement *hooks* and *extensions*. It is part of `pytest`'s infrastructure.
 - **Plugin**: an add-on package that extends or modifies `pytest`'s behaviour.
 - **Hook**: a predefined point in `pytest`'s execution where plugins can run their own code.
 - **Extension**: the extra capability or behaviour added by a plugin, like `async-test` support or new fixtures.
 - **Fixture**: reusable "setup"/initialisation data or resources that `pytest` can automatically provide to a test, such as a temporary directory, database connection, or pre-configured object (e.g., a dictionary that several tests depend upon).

- `anyio-4.10.0` : `AnyIO` is found to have already installed a registered `pytest` plugin (usually supplied in package metadata, like a `pyproject.toml`) as part of the `anyio` package, mainly to support testing asynchronous code.
- `zarr-3.2.1` : Zarr has also installed a `pytest` plugin, likely providing Zarr-related testing support/fixtures.

Learn how to assemble these components:

- writing, discovery, and loading plugins: https://docs.pytest.org/en/latest/how-to/writing_plugins.html
- hook functions and how plugins implement them: https://docs.pytest.org/en/stable/how-to/writing_hook_functions.html
- fixture explanation and usage: <https://docs.pytest.org/en/stable/explanation/fixtures.html>
- `pluggy` : https://pluggy.readthedocs.io/en/latest/api_reference.html
- Python packaging mechanism behind plugin registration and entry points: <https://packaging.python.org/en/latest/guides/creating-and-discovering-plugins>

Further, `pytest` can process `unittest` structures automatically, too.

```
testmod
├── testmod
│   ├── __init__.py
│   └── math_utils.py
└── tests
    └── test_math_utils.py
```

The `./testmod` directory contains a simple package version of the `unittest` example, split into a few pieces. Additionally, we have added a `pytest`-compatible function, for a total of **4 tests**.

Moving into this directory, we can run:

```
python -m pytest
```

This will look for and run **any compatible tests** in that package space.

- `unittest` tests are also included in its search!

```
In [ ]: %cd testmod
        !python -m pytest
        %cd ..
```

TDD Walkthrough with `pytest`

We would like to configure a function called `uppercase_string` via TDD.

Where do we start?

```
In [ ]: #???
```

We have already explored some very basic Input/Output constructs:

- `input()` - `stdin`
- `print()` - `stdout`

Real-world applications **read data** from files, sensors, or user input and **write out** results or logs.

- Without input, most programs are static.
- Without output, users have no feedback or no products are retained.

Basic File Operations

Reading and writing to a file with `open()`.

- <https://docs.python.org/3.13/library/functions.html#open>

Modes of Operation:

`"r"` : **Read (default)**. Opens a file for reading, error if the file does not exist.

`"w"` : **Write**. Opens a file for writing, creates the file if it does not exist.

`"a"` : **Append**. Opens a file for appending, creates the file if it does not exist.

`"x"` : **Create**. Exclusively creates the specified file, returns an error if the file exists.

`"b"` : Binary mode (e.g., images).

`"t"` : Text mode (*default*).

`"+"` : Open for **updating** (reading and writing)

Always close the file after performing operations to free up system resources.

- `.close()` method on file object.
- If we forget to close the file, we may end up with locked files or memory leaks.

Reading a file

```
In [ ]: # Open a file in read mode
file = open("./data/sample_text.txt", "r")

# Read the contents (as one string)
contents = file.read()
print(contents)

# Always close the file manually
file.close()
```

Let's try this with a bit more control:

```
In [ ]: import time
```

```
file = open('./data/sample_text.txt', 'r')

try:
    line = file.readline()
    while line: # Continue reading until the end of the file
        print(line.strip())
        line = file.readline()
        time.sleep(1)
finally:
    print('Closing File.')
    file.close()
```

Writing to a file

- Note using 'w' overwrites the file if it already exists!

```
In [ ]: # Open or create a file in write mode
file = open("sample_write.txt", "w")

# Overwrite existing contents (if any) with new content
file.write("Hello, World!")

# Close the file
file.close()
```

```
In [ ]: # **System specific** newline characters are automatic with print()

file = open('sample_write_newline.txt', 'w')
lines = ['First line', 'Second line', 'Third line']
for line in lines:
    print(line, file=file) # Automatically adds a newline between list entries
file.close()
```

```
In [ ]: %pycat sample_write.txt
```

```
In [ ]: %pycat sample_write_newline.txt
```

Appending to a file

```
In [ ]: # Open a file in append mode
file = open("sample_write_newline.txt", "a")

# Add lines to the end of the file
file.write("Appending a new line.\n")

# Close the file
file.close()
```

```
In [ ]: %pycat sample_write_newline.txt
```

Context Managers

For more professional code, always close your files...or better yet...use **context managers** to automate this process, reducing the risk of file-handling errors.

- Automatic setup and cleanup of resources (files, network connections).

- Ensures proper resource release even if errors are encountered.

```
with open(...) as file:  
    ...
```

```
In [ ]: # Writing to a file  
with open("output.txt", "w") as file:  
    file.write("Let's write a line to a file!\n")  
  
# Reading from a file  
with open("output.txt", "r") as file:  
    contents = file.read()  
  
print(contents)
```

Parenthesized Context Managers

In Python >= 3.10, we are allowed a more readable style if you have **multiple files/resources**. See [PEP 634](#).

- Parentheses around the `with` statements are a syntactic convenience.
 - Enables multiple lines of code for clarity.
- This enables us to do things like tidily read and write at the same time.

```
In [ ]: with (  
    open("sample_write_newline.txt", "r") as f1,  
    open("sample_write.txt", "r") as f2  
):  
    data1 = f1.read()  
    data2 = f2.read()  
  
print(f'{data1=}')  
print(f'{data2=}')
```

Data Files

Data files are central to many real-world Python applications, whether the goal is to:

- Read sensor data for scientific analysis
- Store user preferences in a configuration file
- Transfer data between different systems in a known format

Today, we will look at the **JSON** and **CSV** formats. While these are two of the most common you might encounter, keep in mind that there are MANY others, e.g.:

- XML: a structured format like JSON, becoming less common
- TSV: like CSV, but tab-separated instead of comma-separated values
- Binary formats: often used in scientific contexts (e.g., `pickle`, HDF5, NetCDF)

JSON Files

JSON (JavaScript Object Notation) is a lightweight data format, **structured** like Python dictionaries/lists (though with some syntax differences).

JSON is commonly used for configuration files (environment and application settings), web APIs, and data interchange (send/receive) because it is both human-readable and machine-friendly.

Also:

- Machine learning and data science metadata and hyperparameter storage
- Code logs
- NoSQL databases like MongoDB or CouchDB use JSON-like structures for data storage through tools like `pymongo`

Python has a built-in `json` module that makes it easy to read (**deserialize**) and write (**serialize**) JSON.

Below is a workflow designed to store and load application settings or user data.

```
In [ ]: import json

# Example dictionary that might serve as application config
config_data = {
    "version": "2.0",
    "settings": {
        "theme": "light",
        "auto_save": True,
        "max_retries": 3
    }
}

# Writing to JSON
with open("config.json", "w") as config_file:
    json.dump(config_data, config_file, indent=2)
    # 'indent=2' makes the file more readable

# Reading from JSON
try:
    with open("config.json", "r") as config_file:
        loaded_config = json.load(config_file)
        print("Loaded config:", loaded_config)
except FileNotFoundError:
    print("config.json not found. Falling back to default settings.")
except json.JSONDecodeError:
    print("config.json is malformed JSON. Check file content!")
```

How do we access the `theme` setting?

```
In [ ]: #???
```

Again, use **context managers** to ensure the file is closed automatically.

Explore the rendering of the output file in JupyterLab.

You can handle different exceptions (e.g., `FileNotFoundError`, `json.JSONDecodeError`) to degrade gracefully if the file is missing or corrupted.

To **integrate with the testing concept**, we could write a test that checks if `load_config` returns a dictionary with the correct keys.

CSV Files

The **CSV (Comma-Separated Values)** format is a simple text format often used for spreadsheet-like or **tabular** data.

- Each line typically represents a **row** in a table, and each comma (or other **delimiter**) separates columns.

We will explore two approaches:

1. Reading/writing CSV **manually** by treating it as plain text.
2. Using the standard library `csv` module, which handles edge cases like quoted fields and different delimiters.

You have a file named `weather_data.csv` in the `./data/` directory.

Let's open this in JupyterLab.

- Right-click the file and select "Open With..." to compare the rendering in the CSV Viewer and in the Editor form.
- You should see a **header** row and several rows of data.

Manual Reading

```
In [ ]: try:
    with open("./data/weather_data.csv", "r") as f:
        lines = f.readlines() # List of lines
except FileNotFoundError:
    print("File weather_data.csv not found.")
    lines = []

if lines:
    # Separate columns by comma and store them
    # E.g., ["Date, Temperature, Precipitation", "2024-01-01, 5.0, 0.2", ...]
    header = lines[0].strip().split(",") # ["Date", "Temperature", "Precipitation"]
    data_rows = []

    for line in lines[1:]: # skip header
        parts = line.strip().split(",")
        # parts = ["2024-01-01", "5.0", "0.2"] for instance
        data_rows.append(parts)

    # Print out or further process
    print("Header:", header)
    print("First data row:", data_rows[0] if data_rows else "No rows found.")
```

Manual Writing

```
In [ ]: # Create synthetic data
```

```

# Column header names
new_header = ["Date", "Temperature", "Precipitation"]
# Data for each observation
new_data = [
    ["2024-01-04", "2.5", "0.3"],
    ["2024-01-05", "1.0", "0.0"]
]

# Append to or create a CSV file
with open("./out_synth/weather_data_synthetic.csv", "a") as f:

    # join the elements in the list, separated by a comma, add newline
    f.write(",".join(new_header) + "\n")

    for row in new_data:

        # Join columns by commas, add a newline
        f.write(",".join(row) + "\n")

```

```
In [ ]: %pycat ./out_synth/weather_data_synthetic.csv
```

Notes on manual handling:

- Pros: Simple approach for small files or unique formatting.
 - Cons: Edge cases, such as values containing commas, quotes, or newlines can be cumbersome.
-

Reading and Writing CSV with `csv` module

The built-in `csv` module simplifies dealing with quoted strings, escape characters, different delimiters, etc.

- <https://docs.python.org/3/library/csv.html>
 - Default delimiter is a comma (obvs!), but we can specify others, like `delimiter='\t'`.
 - Default quoted strings use double quotes (`"`), but we can specify others, like `quotechar='|'`.
 - Usually the algorithm that writes CSV will use quotes when a field contains a delimiter, newline or a quote.
 - On reading, the `escapechar` removes any special meaning from the character that follows it.
-

Notes:

- `csv.reader()` returns a **reader object (like an iterator/generator)** to process each row.
- `next(csv_reader)`, when used immediately, grabs the first row, which we treat as the header.
 - Will access subsequent lines individually with each usage, e.g., in a loop.
- `newline=""` is often recommended in the `open()` to avoid extra blank lines, which can result from the default action of auto-conversion of newline characters, on some platforms.

```
In [ ]: import csv

try:
    with open("./data/weather_data.csv", "r", newline="") as f:
        csv_reader = csv.reader(f)
        header = next(csv_reader) # first line is the header
        data_rows = list(csv_reader) # read the rest
except FileNotFoundError:
    print("File weather_data.csv not found.")

```

```
header, data_rows = [], [] # would yield empty lists

print("Header:", header)
print("Data Rows:", len(data_rows))
```

```
In [ ]: # Alternate header/data split with the starred operator

try:
    with open("./data/weather_data.csv", "r", newline="") as f:
        csv_reader = csv.reader(f)
        header, *data_rows = csv_reader # first line is the header
except FileNotFoundError:
    print("File weather_data.csv not found.")
    header, data_rows = [], [] # would yield empty lists

print("Header:", header)
print("Data Rows:", len(data_rows))
```

How can we report the data range (i.e., start and end dates) for this file?

```
In [ ]: #???
```

```
In [ ]: # Writing to CSV...

# Different synthetic data.
# Reuses `new_header` from earlier cell.

new_data = [
    ["2024-01-06", "0.0", "2.1"],
    ["2024-01-07", "-1.0", "1.2"]
]

# Create or overwrite existing csv file (i.e., not appending)
with open("./out_synth/weather_data_synthetic_2.csv", "w", newline="") as f:
    csv_writer = csv.writer(f)
    csv_writer.writerow(new_header)
    for row in new_data:
        csv_writer.writerow(row)
```

```
In [ ]: %pycat ./out_synth/weather_data_synthetic_2.csv
```

```
In [ ]: # ALTERNATIVELY, write all data rows at once:

# Reuses `new_header` from earlier cell.
# Calls on `.writerows`

new_data = [
    ["2024-01-06", "0.0", "2.1"],
    ["2024-01-07", "-1.0", "1.2"]
]

# Create or overwrite existing csv file (i.e., not appending)
with open("./out_synth/weather_data_synthetic_3.csv", "w", newline="") as f:
    csv_writer = csv.writer(f)
    csv_writer.writerows([new_header] + new_data)
```

Comparing Manual vs. `csv` Module

Manual:

- Handle `.split(",")` explicitly.
- Handle special characters in data (e.g., commas in text fields) yourself.
- Good for quick-and-dirty tasks or **non-standard** CSV-like formats.

csv module:

- **Automatically** handles escaping, quotes, and configurable delimiters.
 - More robust for general CSV files, especially if sharing data with Excel, Google Sheets, or other systems.
 - Still quite straightforward to use.
-

Post-Exceptions Summary

- Test early and often.
 - Use TDD when possible.
 - `unittest` and `pytest` are both useful testing tools.
 - We covered reading/writing files and using context managers for safe resource handling.
-

References and Resources

The main guide to errors in Python.

- Python Documentation, 8. Errors and Exceptions, <https://docs.python.org/3.13/tutorial/errors.html>
 - **Note Python version 3.13.**

Nice revisions of much of today's exception material:

- Cassingena Navone, Estefania. How to **Handle Exceptions** in Python: A Detailed Visual Introduction, <https://www.freecodecamp.org/news/exception-handling-python/>
- teclado. Day 24: Advanced Exception **Handling and Raising** Exceptions, <https://teclado.com/30-days-of-python/python-30-day-24-exceptions-advanced/>
- van de Klundert, Said. **Python Exceptions**: An Introduction, <https://realpython.com/python-exceptions/>

More on **Context Managers**:

- Pozo Ramos, Leodanis. Context Managers and Python's `with` Statement, <https://realpython.com/python-with-statement/>
-

Reference:

- Lutz, M. (2025). Learning Python: Powerful Object-Oriented Programming
 - **Chapter 9 and Part VII**
-

FURTHER APPLICATIONS TO EXPLORE:

- File Operations: Handle exceptions that occur during file operations. (See practical)
 - Graceful Degradation: Strategies for gracefully handling exceptions in applications. (See practical)
 - Logging Exceptions: The `logging` module for recording exceptions. (We will return to this topic)
 - <https://docs.python.org/3.13/library/logging.html>
 - **As of Python 3.11:** Exception Groups to handle multiple exceptions at once, particularly useful in concurrent or parallel code.
 - <https://peps.python.org/pep-0654/>
 - **More `pytest` features:** Fixtures, marks, parameterizations, and duration reports.
<https://realpython.com/pytest-python-testing/>
 - Explore the `pathlib` library to efficiently and elegantly handle file path processing:
<https://docs.python.org/3/library/pathlib.html>
 - Explore the `tox` package for testing in multiple environments: <https://pypi.org/project/tox/>
-

In This Week's Practical

Practice with:

1. Exception handling
 2. Testing
 3. Reading and writing files
 4. Regular expressions (**new**)
 5. Developing CLI tools (**see homework**)
-

Next Week

Data Exploration with Pandas and NumPy